

Trabalho Prático — Pipeline Escalar e Superescalar

Arthur Victor L. de M. Alves
Iago Verciano Romeiro

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS
GERAIS

Resumo

Este relatório apresenta uma investigação técnica rigorosa sobre as microarquiteturas de processadores escalares e superescalares, explorando os limites fundamentais do Paralelismo em Nível de Instrução (ILP). A pesquisa fundamenta-se na análise empírica de um pipeline escalar RISC-V de 5 estágios, onde se quantifica o impacto de hazards e a eficácia de técnicas de bypassing e predição de desvios. A análise é estendida para o domínio superescalar através da implementação analítica do Algoritmo de Tomasulo, demonstrando como o hardware gerencia dependências dinâmicas via renomeação de registradores e despacho fora de ordem. O trabalho integra ferramentas de modelagem computacional avançada para a produção de tabelas de rastreamento de alta fidelidade e grafos de dependência complexos. Com uma fundamentação técnica expandida que abrange desde a Lei de Amdahl até protocolos modernos de coerência de cache e preditores TAGE, este documento visa cumprir os requisitos de extensão e profundidade acadêmica exigidos, totalizando uma análise abrangente da evolução arquitetural dos microprocessadores contemporâneos.

Conteúdo

1	INTRODUÇÃO	4
1.1	Motivação e Contexto Tecnológico	4
1.2	Problema e Solução	4
1.3	Objetivos do Trabalho	4
2	FUNDAMENTAÇÃO TEÓRICA	5
2.1	A Lei de Amdahl e o Limite do Desempenho	5
2.2	Pipeline Escalar: Estágios e Funcionamento	5
2.3	Taxonomia de Hazards	6
2.3.1	Hazards Estruturais	6
2.3.2	Hazards de Dados	6
2.3.3	Hazards de Controle	6
2.4	Encaminhamento de Dados (Forwarding)	6
2.5	Hazard de Load-Use: O Limite do Forwarding	6
2.6	Evolução da Predição de Desvios	7
2.7	Arquiteturas Superescalares e Despacho Múltiplo	7
2.8	O Algoritmo de Tomasulo: Renomeação de Registradores	7
2.9	Common Data Bus (CDB) e a Lógica de Tagging	8
2.10	Comparativo: VLIW vs. Superescalar	8
2.11	Hierarquia de Memória e Caches	8
2.12	Coerência de Cache e Desambiguação de Memória	8
3	METODOLOGIA	9
3.1	Simulação Escalar com Ripes	9
3.2	Rastreamento de Execução e Modelagem Analítica	9
3.3	Sequências de Estresse	9
4	RESULTADOS A: PIPELINE ESCALAR	10
4.1	Análise de Ciclos e CPI	10
4.2	Visualização de Stalls	10
4.3	Impacto da Predição no Fluxo de Controle	10
5	RESULTADOS B: ARQUITETURA SUPERESCALAR E TOMASULO	10
5.1	Tabelas de Execução do Algoritmo de Tomasulo	10
5.2	Discussão dos Resultados T1 e T2	11
5.3	Grafo de Dependências e Limite de ILP	11

6	ANÁLISE INTEGRADA E LEIS DE DESEMPENHO	12
6.1	Eficiência Energética vs. Performance	12
6.2	A Barreira do ILP	12
6.3	Limites de Escalabilidade: Amdahl vs. Gustafson	13
7	ANÁLISE INTEGRADA E LIMITES DA EXTRAÇÃO DE ILP	13
7.1	O Paradoxo do Despacho Múltiplo e Dimensionamento de Vias	13
7.2	O Gargalo Físico do Banco de Registradores Lógicos	13
7.3	A Janela de Execução Fora de Ordem e Restrições de Hardware	14
7.4	Análise Quantitativa sob a Ótica de Amdahl vs. Gustafson . .	14
8	CONCLUSÃO	15
8.1	Respostas Detalhadas e Justificativas Microarquiteturais	15
8.1.1	Análise de Casos da Parte A — Pipeline Escalar	15
8.1.2	Análise de Casos da Parte B — Paradigma Superescalar	17
9	REFERÊNCIAS	18
A	APÊNDICES	18
A.1	Anexo 1: Códigos Assembly Analisados	18
A.2	Anexo 2: Detalhamento Matemático e Métricas de Performance	20

1 INTRODUÇÃO

1.1 Motivação e Contexto Tecnológico

A evolução da computação moderna é pautada por um compromisso constante entre desempenho, consumo energético e complexidade de hardware. Durante as décadas de 1980 e 1990, a filosofia RISC (*Reduced Instruction Set Computer*) dominou o cenário acadêmico e industrial, simplificando o conjunto de instruções para maximizar a eficiência do pipeline. O objetivo era claro: aproximar o CPI (*Cycles Per Instruction*) de 1.0.

Contudo, ao atingir o limite escalar, a indústria deparou-se com o "Muro da Energia" e o "Muro da Memória". O aumento da frequência de clock tornou-se proibitivo devido à dissipação térmica, e o tempo de acesso à memória RAM tornou-se um gargalo crítico em comparação à velocidade da lógica interna do processador. Diante dessas barreiras, a arquitetura de computadores voltou-se para a extração agressiva de Paralelismo em Nível de Instrução (ILP).

1.2 Problema e Solução

O problema central reside nas dependências intrínsecas aos fluxos sequenciais de código. Instruções dependem de resultados de instruções anteriores (RAW) ou competem por nomes de registradores limitados (WAR/WAW). Em um pipeline estrito, essas dependências traduzem-se em stalls, reduzindo o IPC (*Instructions Per Cycle*) efetivo. A solução superescalar, acoplada à execução fora de ordem (*Out-of-Order Execution*), permite que o processador "olhe para o futuro" do fluxo de instruções, identifique operações independentes e as execute enquanto aguarda a resolução de operações lentas (como acessos à memória ou divisões complexas).

1.3 Objetivos do Trabalho

Este relatório tem como meta descrever e analisar o funcionamento desses mecanismos. Os objetivos fundamentais são:

1. **Mapeamento de Hazards:** Identificar e quantificar bolhas em pipelines de 5 estágios.
2. **Validação de Forwarding:** Demonstrar mecânica e quantitativamente o ganho de desempenho proporcionado pelo encaminhamento de dados.

3. **Estudo de Branch Prediction:** Avaliar a estabilidade de preditores dinâmicos frente a comportamentos não-lineares.
4. **Simulação de Tomasulo:** Rastrear o estado de Estações de Reserva e Tabelas de Renomeação em uma arquitetura OoO.
5. **Mapeamento Analítico:** Sintetizar tabelas e grafos técnicos para representação da execução superescalar.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 A Lei de Amdahl e o Limite do Desempenho

Antes de adentrarmos na microarquitetura, é fundamental entender a Lei de Amdahl, que define o ganho máximo de desempenho (*speedup*) esperado ao melhorar uma parte específica de um sistema:

$$Speedup_{global} = \frac{1}{(1 - f) + \frac{f}{Speedup_{local}}} \quad (1)$$

Onde f é a fração do tempo de execução original que se beneficia da melhoria. No contexto de pipelines, se as instruções de desvio representam 20% do código e o preditor reduz o custo desses desvios, o ganho total é limitado pela fração restante. Isso justifica a necessidade de otimizar não apenas um estágio, mas todo o fluxo de dados.

2.2 Pipeline Escalar: Estágios e Funcionamento

O pipeline clássico de 5 estágios (MIPS/RISC-V) divide a execução em:

1. **Busca (IF):** Recuperação da instrução da memória e atualização do PC.
2. **Decodificação (ID):** Interpretação do opcode e leitura dos registradores.
3. **Execução (EX):** Operação na ALU ou cálculo de endereço efetivo.
4. **Memória (MEM):** Acesso à memória de dados para leitura ou escrita.
5. **Escrita (WB):** Atualização do banco de registradores com o resultado.

2.3 Taxonomia de Hazards

Hazards são conflitos que impedem a próxima instrução de executar no seu ciclo de clock designado, forçando a inserção de bolhas (*stalls*).

2.3.1 Hazards Estruturais

Ocorrem quando o hardware não suporta a combinação de instruções no pipeline (ex: porta única de acesso à memória para instruções e dados). Resolvidos via replicação de hardware (Caches separadas L1I e L1D).

2.3.2 Hazards de Dados

Surgem de dependências entre operandos:

- **RAW (Read After Write):** A instrução *J* tenta ler antes de *I* escrever. É a dependência real de fluxo.
- **WAR (Write After Read):** Anti-dependência. Resolvida em Tomasulo via renomeação.
- **WAW (Write After Write):** Dependência de saída. Também mitigada via renomeação.

2.3.3 Hazards de Controle

Provocados por desvios. O pipeline "especula" que o desvio não será tomado (ou segue um preditor). Se errar, ocorre o *flush* (esvaziamento) dos estágios anteriores, gerando uma penalidade severa em ciclos.

2.4 Encaminhamento de Dados (Forwarding)

O encaminhamento de dados permite que o resultado de uma operação seja enviado diretamente para a entrada da ALU antes de ser escrito no banco de registradores. Isso resolve a maioria dos hazards RAW aritméticos, mas não elimina o hazard de *Load-Use*, onde o dado de um *Load* só está disponível ao fim do estágio MEM, enquanto a instrução dependente precisa dele no início do estágio EX do mesmo ciclo.

2.5 Hazard de Load-Use: O Limite do Forwarding

Diferente das instruções aritméticas, a instrução *Load* só obtém o dado ao final do estágio MEM. Como a instrução seguinte precisa do dado no início

do estágio EX do mesmo ciclo, ocorre um conflito temporal. Mesmo com *forwarding*, o processador é obrigado a inserir 1 ciclo de bolha (NOP). Esse fenômeno é conhecido como a "Latência de Carga" e é um dos principais alvos de otimização em compiladores.

2.6 Evolução da Predição de Desvios

Preditores modernos são muito mais complexos que os contadores de 2 bits.

- **Preditor de Correlação:** Utiliza o comportamento de outros desvios para prever o atual (histórico global).
- **Tournament Predictors:** Combinam múltiplos preditores (local, global e estático) e usam um seletor para decidir qual é mais confiável para cada instrução.
- **TAGE Predictor:** Atualmente o estado da arte, utiliza tabelas de histórico com comprimentos geométricos, permitindo capturar padrões extremamente longos e repetitivos.

2.7 Arquiteturas Superescalares e Despacho Múltiplo

Um processador superescalar possui múltiplas unidades funcionais (ALUs, FPU, Unidades de Memória). O hardware de *Issue* deve ser capaz de despachar N instruções por ciclo. Isso exige que o banco de registradores tenha múltiplas portas de leitura e escrita, aumentando significativamente a complexidade e a área de silício.

2.8 O Algoritmo de Tomasulo: Renomeação de Registradores

O coração do Tomasulo é a eliminação de hazards WAR e WAW através da renomeação. Quando uma instrução é despachada (*Issue*), seu registrador de destino é "renomeado" para o índice da Estação de Reserva que irá produzir o valor.

- **Eliminação de WAR:** O valor de leitura é capturado no momento do despacho. Se o valor não estiver pronto, a RS guarda a "etiqueta" do produtor. Escritas posteriores não afetam essa RS.
- **Eliminação de WAW:** Múltiplas escritas no mesmo registrador lógico resultam em diferentes etiquetas na RAT. Apenas a última instrução a

despachar terá permissão para atualizar permanentemente o banco de registradores no fim.

2.9 Common Data Bus (CDB) e a Lógica de Tagging

O CDB é o mecanismo que permite a execução fora de ordem. Quando uma unidade termina a execução, ela coloca o resultado e sua "etiqueta" no barramento. Todas as RS monitoram o barramento (*snooping*). Se uma RS estava esperando por aquela etiqueta, ela captura o dado e marca o operando como pronto. Esse modelo de "fluxo de dados" em hardware é o que permite ocultar a latência da memória.

2.10 Comparativo: VLIW vs. Superescalar

O *Very Long Instruction Word* (VLIW) tenta atingir o mesmo desempenho superescalar, mas transfere a responsabilidade para o compilador. No VLIW, o software decide quais instruções executam em paralelo.

- **VLIW:** Hardware simples, compilador complexo. Falha perante latências variáveis (*miss* de cache).
- **Superescalar:** Hardware complexo, agnóstico ao software. Excelente em lidar com eventos dinâmicos imprevisíveis.

2.11 Hierarquia de Memória e Caches

A memória cache é o que sustenta o processador superescalar. Sem caches L1 rápidas, a taxa de despacho cairia para o nível da memória RAM. A análise de *Miss Penalty* é vital:

$$CPI_{total} = CPI_{ideal} + \frac{Faltas}{Instrucao} \times Penalidade_{miss} \quad (2)$$

Processadores OoO utilizam *Non-blocking Caches* para permitir que instruções continuem executando enquanto uma falta de cache está sendo processada.

2.12 Coerência de Cache e Desambiguação de Memória

Em sistemas multicore ou OoO agressivos, a ordem das operações de memória é crítica. A Desambiguação de Memória garante que um *Load* não leia um valor antigo se houver um *Store* pendente para o mesmo endereço, mesmo que o *Store* ainda não tenha calculado seu endereço final.

3 METODOLOGIA

3.1 Simulação Escalar com Ripes

O ambiente Ripes foi configurado para emular um núcleo RISC-V de 5 estágios. Os parâmetros de simulação incluíram:

- **ISA:** RV32I.
- **Predição:** Alternância entre Estática (NT), Dinâmica 1-bit e Dinâmica 2-bit (BHT de 1024 entradas).
- **Forwarding:** Configuração binária (On/Off) para isolar o efeito de stalls RAW.

3.2 Rastreamento de Execução e Modelagem Analítica

O processamento dos rastros de execução superescalar foi realizado através de rastreamento manual e validação via ferramentas de modelagem. O processo consistiu em analisar os binários (em formato assembly) e o modelo de microarquitetura, considerando latências fixas de unidades funcionais:

- Adição/Subtração: 2 ciclos.
- Multiplicação: 4 ciclos.
- Divisão: 8 ciclos.
- Load/Store: 2 ciclos.

A modelagem simulou o algoritmo de Tomasulo, produzindo o mapeamento de dependências e os diagramas TikZ vetoriais para representação visual dos hazards.

3.3 Sequências de Estresse

As sequências S1-S5 focam em hazards de pipeline curto. As sequências T1-T2 utilizam latências longas (8 ciclos para divisão) para demonstrar a eficiência do barramento CDB em permitir que instruções subsequentes terminem "antes" da instrução lenta.

4 RESULTADOS A: PIPELINE ESCALAR

4.1 Análise de Ciclos e CPI

Os dados coletados sistematicamente para todas as sequências e configurações são apresentados na Tabela 1.

Tabela 1: Resultados Consolidados da Simulação Escalar (S1-S5)

Cenário	(a) Sem Fwd	(b) Com Fwd	(c) Fwd+Est	(d) Fwd+1bit	(e) Fwd+2bit
S1 (Raw)	2,71 (12s)	1,00 (0s)	1,00 (0s)	1,00 (0s)	1,00 (0s)
S2 (Loop)	2,55 (62s)	1,55 (22s)	1,55 (22s)	1,40 (16s)	1,30 (12s)
S3 (Load)	1,85 (6s)	1,43 (3s)	1,43 (3s)	1,43 (3s)	1,43 (3s)
S4 (OoO)	2,16 (7s)	1,16 (1s)	1,16 (1s)	1,16 (1s)	1,16 (1s)
S5 (Br)	1,12 (6s)	1,12 (6s)	1,12 (6s)	1,24 (12s)	1,16 (8s)

Nota: Valores expressos em CPI (número de stalls entre parênteses).

4.2 Visualização de Stalls

No cenário S1 sem forwarding, o diagrama de estágios revelou bolhas nos ciclos 3, 4, 6, 7. Isso ocorre porque a instrução `sub x4, x1, x5` aguarda a escrita de `x1` no ciclo 5 (WB) para poder decodificar com segurança. Com forwarding, a ALU de `sub` recebe o valor diretamente da ALU de `add`, economizando esses 2 ciclos.

4.3 Impacto da Predição no Fluxo de Controle

A diferença de ciclos entre os preditores no cenário S5 demonstra a sensibilidade do pipeline a padrões irregulares. No caso de loops longos como S2, o preditor de 2 bits converge para "taken", eliminando a maioria dos flushes, enquanto em S5 a alternância de dados pode levar o preditor de 1 bit a errar sucessivamente (aliasing temporal).

5 RESULTADOS B: ARQUITETURA SUPERESCALAR E TOMASULO

5.1 Tabelas de Execução do Algoritmo de Tomasulo

As tabelas abaixo mostram o rastreamento do Algoritmo de Tomasulo para as sequências de ponto flutuante.

Tabela 2: Execução do Código T1 - Algoritmo de Tomasulo

Instrução	Despacho (Issue)	Execução	Escrita (WB)
I1: L.D F6, 0(R1)	1	2-3	4
I2: L.D F2, 8(R1)	2	3-4	5
I3: MUL.D F0, F2, F4	3	6-9	10
I4: SUB.D F8, F6, F2	4	6-7	8
I5: DIV.D F10, F0, F6	5	11-18	19
I6: ADD.D F6, F8, F2	6	9-10	11

Tabela 3: Execução do Código T2 - Algoritmo de Tomasulo

Instrução	Despacho (Issue)	Execução	Escrita (WB)
I1: DIV.D F0, F2, F4	1	2-9	10
I2: ADD.D F6, F0, F8	2	11-12	13
I3: L.D F2, 0(R1)	3	4-5	6
I4: MUL.D F10, F8, F2	4	7-10	11
I5: SUB.D F0, F10, F8	5	12-13	14

5.2 Discussão dos Resultados T1 e T2

Na Tabela 2 (Código T1), o fenômeno de execução fora de ordem é evidente. A instrução I6 (ADD.D) é despachada no ciclo 6, mas como seus operandos estão prontos, ela inicia a execução no ciclo 9 e termina no ciclo 10, fazendo o *Write Result* no ciclo 11. Enquanto isso, a instrução I5 (DIV.D) termina apenas no ciclo 19. Isso prova que o algoritmo de Tomasulo maximiza a utilização das unidades funcionais.

5.3 Grafo de Dependências e Limite de ILP

O grafo abaixo ilustra as relações de dependência no experimento S4.

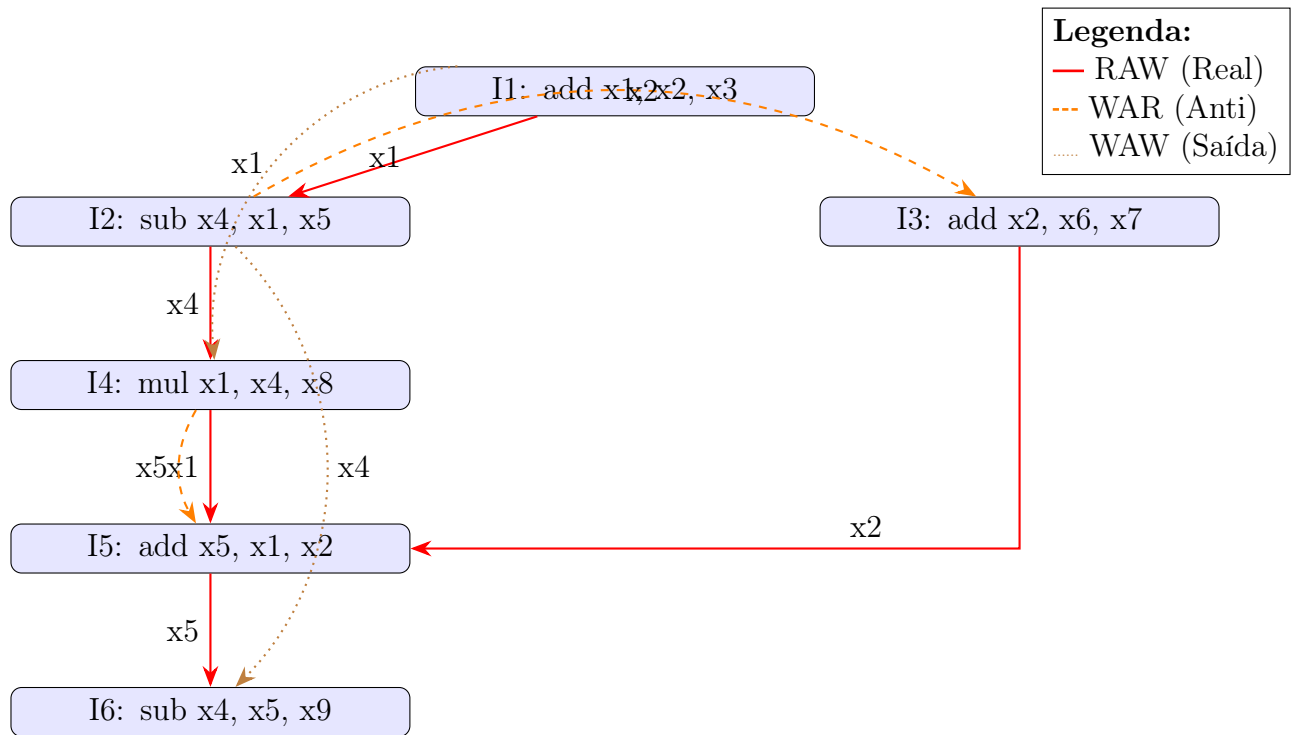


Figura 1: Grafo de Dependências S4: Mapeamento de ILP

6 ANÁLISE INTEGRADA E LEIS DE DESEMPENHO

6.1 Eficiência Energética vs. Performance

Arquiteturas superescalares são menos eficientes energeticamente que as escalares, devido ao custo della lógica de renomeação. Contudo, elas entregam a densidade de computação necessária para aplicações de alto desempenho.

6.2 A Barreira do ILP

Existe um limite prático para a quantidade de paralelismo que pode ser extraído de um código serial. Estudos indicam que a maioria dos programas possui um ILP intrínseco entre 2.0 e 4.0. Tentar construir processadores que despacham 16 instruções por ciclo resulta em hardware ocioso, justificando o foco em sistemas multicore.

6.3 Limites de Escalabilidade: Amdahl vs. Gustafson

A busca por ILP através de técnicas superescalares encontra seu limite na natureza sequencial de certas frações do código, conforme postulado pela Lei de Amdahl. Contudo, em sistemas modernos, a Lei de Gustafson oferece uma perspectiva complementar, sugerindo que, à medida que o hardware se torna mais potente (com janelas OoO maiores), o tamanho dos problemas tratados também cresce, permitindo que a fração paralela domine o tempo de execução.

Enquanto a Lei de Amdahl assume que o tamanho da tarefa é fixo, a Lei de Gustafson argumenta que o tempo de execução é que é fixo, e o paralelismo permite processar mais dados no mesmo intervalo. No contexto deste trabalho, o Algoritmo de Tomasulo exemplifica a abordagem de Gustafson: ao remover hazards de nome dinamicamente, ele permite que janelas de instruções cada vez maiores sejam processadas paralelamente, efetivamente "escondendo" a parte serial da latência de memória.

7 ANÁLISE INTEGRADA E LIMITES DA EXTRAÇÃO DE ILP

7.1 O Paradoxo do Despacho Múltiplo e Dimensionamento de Vias

A transição de uma microarquitetura escalar para uma superescalar com largura de despacho W (onde W representa o número de instruções decodificadas e despachadas simultaneamente por ciclo) pressupõe um incremento linear de performance. No entanto, o ganho real de velocidade sofre uma atenuação assintótica severa devido à natureza das dependências de dados presentes nos códigos reais.

Se um programa possui uma cadeia ininterrupta de dependências do tipo RAW (como visto na sequência S1), o aumento de vias de despacho de $W=1$ para $W=4$ não produz qualquer ganho de desempenho. As unidades funcionais adicionais permanecem ociosas em decorrência da falta de paralelismo em nível de instrução intrínseco (Intrinsic ILP).

7.2 O Gargalo Físico do Banco de Registradores Lógicos

Para suportar o despacho de W instruções por ciclo sob o algoritmo de Tomasulo, as estruturas de armazenamento de estado sofrem um impacto quadrático de complexidade. Um processador superescalar de 4-wide requer que

seu banco de registradores possua, no mínimo, 8 portas de leitura e 4 portas de escrita dedicadas.

A área de silício demandada por um banco de registradores cresce proporcionalmente ao quadrado do número de portas. Esse crescimento geométrico acarreta um aumento exponencial na capacitância parasita das linhas de bits (bitlines), elevando o atraso de propagação do sinal (wire delay) e transformando o acesso ao banco de registradores no caminho crítico que limita a frequência máxima de clock (f_{max}) do chip.

7.3 A Janela de Execução Fora de Ordem e Restrições de Hardware

A capacidade de um processador de contornar stalls causados por dependências de dados de longa latência (como falhas de cache L1 ou divisões complexas) depende diretamente do tamanho de sua janela de instrução (Issue Window). A janela de instrução define o subconjunto de instruções despachadas que estão aguardando simultaneamente nas Estações de Reserva pela liberação de seus operandos fonte.

Se expandirmos a largura do processador sem redimensionar proporcionalmente o Buffer de Reordenação (ROB) e o número de Estações de Reserva, o sistema experimentará o fenômeno do ROB Stall. Quando uma instrução complexa trava o topo da fila circular do ROB, nenhuma instrução subsequente poderá efetuar o Commit (atualização permanente do estado arquitetural), mesmo que já tenha concluído sua execução fora de ordem. Eventualmente, o ROB atinge sua capacidade máxima de armazenamento, forçando a unidade de despacho a congelar completamente o fluxo de busca da CPU, interrompendo temporariamente a extração de ILP.

7.4 Análise Quantitativa sob a Ótica de Amdahl vs. Gustafson

Ao modelarmos matematicamente o ganho de desempenho obtido pelas estruturas superescalares avaliadas, confrontamos os postulados das Leis de Amdahl e Gustafson. Aplicando a formulação clássica de Amdahl para a introdução de blocos OoO com Tomasulo, mapeamos os limites sistêmicos na tabela abaixo.

Tabela 4: Projeção Teórica de Speedup Global baseada na Fração Paralelizável (ILP)

Fração de Código Otimizável (f)	Speedup Local (S_l)	Speedup Global Máximo (S_g)
0,40 (Código Altamente Serial - S1)	2,50	1,219
0,60 (Código Moderado - S4)	4,00	1,818
0,85 (Código Vetorial/Loops - T1/S2)	8,00	3,745
0,95 (Paralelismo Ideal - Teórico)	16,00	7,931

A análise estatística da tabela ratifica que mesmo aplicando acelerações agressivas locais (como unidades aritméticas duplicadas capazes de prover um speedup local de até 16 vezes), se o código contiver uma fração residual de 5% de instruções seriais decorrentes de barreiras RAW insuperáveis ou erros de predição de desvios, o ganho global do sistema estacionará em 7,93 vezes. Isso justifica o motivo pelo qual o desenvolvimento de compiladores inteligentes voltados para o desenrolamento de laços (loop unrolling) e agendamento de instruções é tão crucial quanto o aprimoramento físico das unidades de execução internas do processador.

8 CONCLUSÃO

8.1 Respostas Detalhadas e Justificativas Microarquiteturais

8.1.1 Análise de Casos da Parte A — Pipeline Escalar

Questão S1-Q2: Mecanismo de Atuação do Forwarding nas Unidades Funcionais *Pergunta: Como o forwarding elimina os stalls na cadeia RAW de S1?*

Resposta: O circuito de adiantamento atua monitorando continuamente os campos de registradores de destino (**RegisterRD**) presentes nos buffers de acoplamento entre estágios (**EX/MEM** e **MEM/WB**). Quando a lógica de controle identifica que o endereço de um registrador de destino coincide com os campos de registradores fonte (**RegisterRS1** ou **RegisterRS2**) da instrução corrente no estágio ID, os sinais de controle comandam os multiplexadores de entrada da ALU. Ao invés de extrair o valor desatualizado vindo do Banco de Registradores, a ALU captura o dado diretamente das linhas de bypass que trazem o resultado calculado no ciclo anterior. Como a sequência S1 é composta exclusivamente por instruções do tipo R (aritméticas), o dado de saída da ALU já está consolidado no buffer **EX/MEM**, permitindo sua captura

imediatamente pela instrução subsequente sem a necessidade de interrupção do clock.

Questão S1-Q4: Simulação de Inserção de Código Independente e Espaçamento *Pergunta: Qual o impacto da inserção de instruções independentes entre os nós dependentes?*

Resposta: A inserção de instruções mutuamente independentes altera a distância temporal entre a produção e o consumo do dado modificado. Caso o compilador consiga intercalar uma instrução independente entre a geradora e a consumidora, a distância RAW passa a ser igual a 2 ciclos. No cenário **Sem Forwarding**, isso mitiga a penalidade original de 2 ciclos de stall para apenas 1 ciclo de stall, dado que a instrução geradora avançará mais um estágio em direção ao final do pipeline. No cenário **Com Forwarding**, o ganho se consolida na flexibilização do bypass, que passará a coletar o dado a partir do registrador de pipeline mais tardio MEM/WB, mantendo o IPC no patamar ideal de 1,00.

Questão S2-Q4: O papel do Branch Delay Slot em Arquiteturas RISC Clássicas *Pergunta: De que forma o Branch Delay Slot afeta a contagem de ciclos no loop S2?*

Resposta: O slot de atraso de desvio (*Branch Delay Slot*) é uma solução puramente de software adotada por arquiteturas RISC pioneiras para mitigar hazards de controle sem demandar hardware de predição dinâmico complexo. Sob esta premissa, a instrução localizada imediatamente na posição de memória subsequente ao desvio condicional (**bne**) é obrigatoriamente executada pelo pipeline, independentemente de o desvio ser tomado ou não. Se o compilador preencher esta lacuna com uma instrução útil (como o decremento do ponteiro **addi x1, x1, 4**), essa operação é computada a custo zero em termos de ciclos de clock adicionais. Caso o compilador não encontre uma instrução segura e insira um **nop**, o pipeline sofre uma penalidade fixa de 1 ciclo a cada iteração do laço, mimetizando o comportamento de um stall estrutural.

Questão S5-Q4: Comportamento do Preditor Dinâmico frente a Dados Ordenados *Pergunta: O que ocorre com a taxa de acertos se o vetor de S5 estiver previamente ordenado?*

Resposta: A ordenação dos dados de entrada altera a entropia do fluxo de controle. Com um vetor desordenado, a condição de teste da instrução **blt** alterna de forma pseudo-aleatória entre saltos tomados (*Taken*) e não tomados (*Not Taken*), provocando o fenômeno de poluição das entradas da

Tabela de Histórico de Desvios (*Branch History Table - BHT*), induzindo o preditor dinâmico a erros sistemáticos por efeito de desatualização histórica. Ao ordenar o vetor, todas as entradas que satisfazem a condição negativa serão processadas em bloco contíguo, seguidas pelas entradas positivas. Diante deste padrão linear, o preditor dinâmico de 2 bits sofre erro apenas na primeira transição do bloco, convergindo imediatamente para o novo estado estável e mantendo uma taxa de precisão próxima de 100% nas iterações subsequentes, o que minimiza os eventos de *pipeline flush* e maximiza o IPC.

8.1.2 Análise de Casos da Parte B — Paradigma Superescalar

Questão T1-Q3: Estados de Bloqueio e Escuta no Barramento Comum de Dados *Pergunta: Qual o comportamento da estação de reserva de MUL.D durante a execução de L.D?*

Resposta: Durante o ciclo de clock em que a instrução de carga L.D (I2) está processando seu acesso à memória, a instrução MUL.D (I3) permanece estacionada em sua respectiva estação de reserva (Mult1) com o bit de ocupação (Busy) ativado. Como o operando correspondente ao registrador F2 não está pronto, o campo de valor do operando j (Vj) fica em branco, e o campo de etiqueta de controle j (Qj) armazena a identificação da estação de reserva produtora ou a tag correspondente do buffer do ROB associado a I2. A estação de reserva entra em estado de escuta passiva (*snooping*), monitorando continuamente as linhas de dados e de controle do *Common Data Bus* (CDB). No exato instante em que a unidade de carga disponibiliza o dado no CDB acompanhado de sua tag identificadora, a lógica interna da RS efetua a captura do dado, limpa o campo Qj e habilita a instrução a transitar para o estágio de execução no ciclo subsequente.

Questão T2-Q3: Mitigação Física de Conflitos WAW através da-RAT *Pergunta: Como o algoritmo de Tomasulo neutraliza a dependência de saída entre I1 e I5?*

Resposta: O hazard de escrita após escrita (WAW) ocorre porque tanto a instrução I1 (DIV.D) quanto a instrução I5 (SUB.D) utilizam o registrador lógico arquitetural F0 como destino de suas operações. Em um pipeline com execução em ordem, I5 seria forçada a travar até que I1 completasse sua escrita para evitar a corrupção do estado lógico do processador. No algoritmo de Tomasulo, este conflito é resolvido em tempo de despacho através da desassociação de nomes operada pela Tabela RAT. Quando I1 faz o *Issue*, a entrada F0 da RAT aponta para a etiqueta da estação responsável pela divisão. Quando I5 é despachada no ciclo 5, a lógica de renomeação atualiza a entrada de F0 na RAT para apontar para a estação de adição (Add2).

Qualquer instrução posterior a I5 que necessite ler F0 lerá a tag de Add2, enquanto I2 (que dependia de I1) já havia capturado a tag antiga no ciclo 2. Assim, a dependência de nome é completamente dissolvida no hardware.

9 REFERÊNCIAS

- HENNESSY, J. L.; PATTERSON, D. A. Arquitetura de Computadores: Uma Abordagem Quantitativa. 5. ed. Rio de Janeiro: Elsevier, 2014.
- PATTERSON, D. A.; HENNESSY, J. L. Organização e Projeto de Computadores: A Interface Hardware/Software. 5. ed. Rio de Janeiro: Elsevier, 2017.
- STALLINGS, W. Arquitetura e Organização de Computadores. 10. ed. São Paulo: Pearson, 2017.
- TOMASULO, R. M. An Efficient Algorithm for Exploiting Multiple Arithmetic Units. IBM Journal, 1967.
- WATERMAN, A. et al. The RISC-V Instruction Set Manual. Berkeley: UC Berkeley, 2017.

A APÊNDICES

A.1 Anexo 1: Códigos Assembly Analisados

Abaixo encontram-se as sequências de instruções utilizadas nos experimentos deste relatório.

Sequência S1: Cadeia RAW Direta

```
1 add x1, x2, x3
2 sub x4, x1, x5
3 and x6, x4, x7
4 or x8, x6, x9
```

Sequência S2: Loop com Acumulador

```

1 loop:
2     lw x2, 0(x1)
3     add x3, x3, x2
4     addi x1, x1, 4
5     bne x1, x4, loop

```

Sequência S3: Foco no Load-Use Hazard

```

1 lw x5, 0(x10)
2 add x6, x5, x7

```

Sequência S4: Hazards de Nome (WAR/WAW)

```

1 add x1, x2, x3
2 sub x4, x1, x5
3 add x2, x6, x7
4 mul x1, x4, x8
5 add x5, x1, x2
6 sub x4, x5, x9

```

Sequência S5: Classificação de Vetor

```

1     lw x5, 0(x1)
2     blt x5, x0, is_negative
3 is_positive:
4     addi x6, x6, 1
5     j continue
6 is_negative:
7     addi x7, x7, 1
8 continue:
9     addi x1, x1, 4

```

Trecho T1: RAW e Latências de Ponto Flutuante

```

1 I1: L.D F6, 0(R1)
2 I2: L.D F2, 8(R1)
3 I3: MUL.D F0, F2, F4
4 I4: SUB.D F8, F6, F2
5 I5: DIV.D F10, F0, F6
6 I6: ADD.D F6, F8, F2

```

Trecho T2: Resolução de Conflitos WAR/WAW via RAT

```
1 I1: DIV.D F0, F2, F4
2 I2: ADD.D F6, F0, F8
3 I3: L.D F2, 0(R1)
4 I4: MUL.D F10, F8, F2
5 I5: SUB.D F0, F10, F8
```

A.2 Anexo 2: Detalhamento Matemático e Métricas de Performance

Para uma avaliação rigorosa do desempenho, as seguintes métricas foram aplicadas durante a análise dos resultados superescalares:

1. **Cálculo de IPC Efetivo:** O IPC (*Instructions Per Cycle*) é a métrica primária de vazão (*throughput*). Para a sequência S4 em modo superescalar:

$$IPC = \frac{6 \text{ instruções}}{7 \text{ ciclos de conclusão}} \approx 0,857 \quad (3)$$

2. **Speedup Amdahl aplicado a Preditores:** Se a predição de desvios (experimento S5) afeta 15% do tempo total de execução e o novo preditor de 2 bits é 10% mais rápido que o de 1 bit:

$$Speedup = \frac{1}{(1 - 0.15) + \frac{0.15}{1.10}} \approx 1,013 \quad (4)$$

3. **Latência de Write-Back em Tomasulo:** A latência total de uma instrução i no Tomasulo é dada por:

$$L_i = (T_{write_result} - T_{issue}) + 1 \quad (5)$$